



OP3 Arm-Only Brace Environment System Design

Overview

The OP3 Arm-Only Brace environment is a simplified variant of the full OP3 Brace task. The robot learns to control **only its arms** to brace itself when pushed, without any hip or knee actuation. The legs remain rigid (fixed in place), and all bracing effort must come from arm extension. This is a more constrained, arm-focused variant useful for studying upper-body impact absorption.

Key Design Principle: No Warp GPU parallelization; single-env, pure MuJoCo CPU simulation with PyTorch for learning. Arm-only control; legs locked at zero angles.

Observation Space

Dimension: 19 (continuous, clipped to $[-50, +50]$ per element)

Index	Name	Unit	Description
0	body_roll	rad	Body roll angle (rotation around X)
1	body_pitch	rad	Body pitch angle (rotation around Y)
2-3	body_angvel_rp	rad/s	Angular velocity (roll rate, pitch rate)
4-9	joint_angles	rad	6 actuated arm joint positions: [l_sho_pitch, l_sho_roll, l_el, r_sho_pitch, r_sho_roll, r_el]
10-15	joint_velocities	rad/s	6 actuated arm joint velocities: [l_sho_pitch_vel, l_sho_roll_vel, l_el_vel, r_sho_pitch_vel, r_sho_roll_vel, r_el_vel]
16-18	contact_flags	{0,1}	3 binary contact indicators: [arms_contact, head_contact, floor_contact] MuJoCo contact with floor, 0 = no contact)

Notes

- **Body orientation** extracted from the root (free) joint quaternion via Euler angle conversion.
 - **Body angular velocity** (roll, pitch rates) extracted from body frame angular velocity.
 - **Arm joint angles and velocities** are absolute positions and velocities for the 6 shoulder/elbow joints.
 - **Contact flags:** arms_contact (arms hit floor), head_contact (head hits floor = failure), feet_contact (feet/legs hit floor).
 - **Contact detection:** Via MuJoCo contact array (`data.ncon` , `data.contact[i]`).
-

Action Space

Dimension: 6 (continuous, normalized to $[-1, 1]$)

Normalized actions are mapped to absolute joint position targets via affine scaling:

$$\text{target}_j = \text{ctrl_min}_j + 0.5 \cdot (\text{action}_j + 1.0) \cdot (\text{ctrl_max}_j - \text{ctrl_min}_j)$$

Actuated Joints (Arms Only)

Index	Joint Name	Control Range [rad]	Notes
0	<code>l_sho_pitch</code>	$[-1.7, 1.7]$	Left shoulder pitch
1	<code>l_sho_roll</code>	$[-1, 1]$	Left shoulder roll
2	<code>l_el</code>	$[-1, 1]$	Left elbow
3	<code>r_sho_pitch</code>	$[-1.7, 1.7]$	Right shoulder pitch
4	<code>r_sho_roll</code>	$[-1, 1]$	Right shoulder roll
5	<code>r_el</code>	$[-1, 1]$	Right elbow

Fixed (Non-Actuated) Joints

Joint	Value [rad]	Notes
head_pan	0.0	Always straight ahead
head_tilt	-1.0	Always fixed down
l_hip_yaw	0.0	Locked
r_hip_yaw	0.0	Locked
l_hip_roll	0.0	Locked
r_hip_roll	0.0	Locked
l_hip_pitch	0.0	Locked
r_hip_pitch	0.0	Locked
l_knee	0.0	Locked
r_knee	0.0	Locked
l_ank_pitch	0.0	Locked
l_ank_roll	0.0	Locked
r_ank_pitch	0.0	Locked
r_ank_roll	0.0	Locked

Implementation: During environment step, all fixed joints are explicitly set to their target values in the control input before calling `mujoco.mj_step()`.

Policy Network Architecture

Class: `Op3ArmPP0Network(nn.Module)`

```
Input: obs (shape [batch] or [])
↓
Shared Backbone:
  Linear(19 → 64) + Tanh
  Linear(64 → 64) + Tanh
↓
Policy Head:
  Linear(64 → 6) → mean (action means)

Value Head:
  Linear(64 → 1) → value (scalar critic estimate)

Learned Parameters:
  policy_logstd: Parameter(shape=[6]) (log std of Gaussian policy)
```

Policy Distribution: Gaussian with state-dependent mean and learned fixed variance (diagonal covariance).

Training Hyperparameters

All hyperparameters are inherited from the `brace_only_single` baseline to ensure consistency.

Category	Parameter	Value
Optimization	Learning rate	3e−4 (Adam)
	Gradient clipping	0.5 (L2 norm)
PPO	Clipping range (ϵ)	0.2
	Value loss coefficient	0.5
	Entropy coefficient (initial)	0.01

Category	Parameter	Value
Exploration	Entropy decay (per iter)	0.995
	Minimum entropy coef	0.001
Sampling	Steps per iteration	2048
	Trajectory GAE λ	0.95
	Discount factor (γ)	0.99
Episode	Episode length (timeout)	200 steps
	MuJoCo substeps per step	5
Stopping	Target success rate	$\geq 95\%$ (over 10-episode window)
	Early stop condition	Maintain $\geq 95\%$ for 100+ consecutive episodes
Checkpointing	Save interval	Every 10 iterations
	Maximum iterations	1000

Reward Function

Per-step reward is computed as a combination of contact-sequence reward, torque penalty, jitter penalty, and a success bonus applied at episode end.

1. Contact Sequence Reward ($R_{contact}$)

Encourages arms to contact the floor first (before head), with synchronized left/right arm impact. **All contact timing based on MuJoCo contact array**

detection.

$$R_{contact} = w_1 \cdot r_{arm_first} + w_2 \cdot r_{arm_sync} - c_{head_impact}$$

Terms:

- r_{arm_first} (weight $w_1 = 1.0$):

If at least one arm has made ground contact (detected via MuJoCo contact array):

$$r_{arm_first} = \tanh\left(\frac{\max(t_H - t_{arms}^{min}, 0)}{1.0}\right)$$

where $t_{arms}^{min} = \min(t_A^L, t_A^R)$ (first arm contact time from MuJoCo array) and t_H is head contact time (from MuJoCo array). If no arms contact, $r_{arm_first} = 0$.

- r_{arm_sync} (weight $w_2 = 0.8$):

Reward for synchronized left/right arm contact (both detected via MuJoCo contact array):

$$r_{arm_sync} = 1.0 - \tanh(|t_A^L - t_A^R|)$$

where t_A^L and t_A^R are first contact times for left and right arms from the MuJoCo contact array. Evaluated only if both arms contact; else $r_{arm_sync} = 0$.

- c_{head_impact} (per-step penalty):

$$c_{head_impact} = \max(0, 0.15 - z_{head}) \times 5.0$$

Penalizes low head position to discourage head-ground proximity. (Soft penalty; hard termination occurs only if MuJoCo detects actual head-ground contact.)

2. Torque Load Penalty (R_{torque})

Per-step penalty on cumulative joint actuation effort to encourage energy-efficient bracing.

$$R_{torque} = - \sum_{j=1}^6 |\tau_j|$$

where:

- τ_j = actuator force/effort of arm joint j , computed via `data.actuator_force[actuator_id]`
- Summed over all 6 actuated arm joints per step (NOT normalized; raw effort sum)

3. Jitter Penalty (R_{jitter})

Per-step penalty on action sign-flips to encourage smooth, coherent movement.

$$R_{jitter} = -n_{jitter}$$

where n_{jitter} is the count of actuators experiencing sign-flip jitter:

- For each of the 6 arm actuators, check if the normalized action value changed sign between consecutive steps.
- **Condition for jitter:** $|\text{action}_{t-1}[j]| \geq 0.5$ AND $|\text{action}_t[j]| \geq 0.5$ AND $\text{sign}(\text{action}_{t-1}[j]) \neq \text{sign}(\text{action}_t[j])$
- Count the number of actuators satisfying this condition; $n_{jitter} \in [0, 6]$.

4. Success Bonus

Applied **only at episode termination** if success criteria are met:

$$R_{success} = \begin{cases} +100.0 & \text{if episode succeeds} \\ 0.0 & \text{otherwise} \end{cases}$$

Total Reward Function

Per-step reward (before success bonus):

$$R_t = w_{contact} \cdot R_{contact}(t) + w_{torque} \cdot R_{torque}(t) + w_{jitter} \cdot R_{jitter}(t)$$

At episode termination:

$$R_{final} = R_{success}$$

Combined episode-step reward:

$$R_{total} = \sum_{t=1}^T R_t + R_{final}$$

where T is the episode length (≤ 200 steps) and weights are:

Component	Weight	Priority	Intuition
$R_{contact}$	$w_{contact} = 10.0$	1st (Highest)	Arm-first contact is the core task objective.
R_{torque}	$w_{torque} = 0.5$	2nd	Energy efficiency is secondary; encourage controlled, low-effort bracing.
R_{jitter}	$w_{jitter} = 0.5$	2nd (Equal)	Smooth movement prevents oscillation and jerky behavior; equal to torque.
$R_{success}$	—	Bonus	Episode-level success bonus applied only if all success criteria met.

Clipping: Final per-step reward is clipped to $[-100, +100]$.

Episode Termination & Success

Termination Conditions (done = True)

Episode ends if **any** of the following occur:

- Head Contact (Failure):** Head collision geom makes actual MuJoCo contact with floor geom.
 - Detection: Iterate `data.contact[0:data.ncon]` and check if any contact pair

includes a head geom (`h1c` , `h2c` , `h21c` , `h22c`) and the floor geom.

- Immediate episode termination, `success = False`.
2. **Timeout:** Step count reaches or exceeds 200 steps.
 - No automatic failure; success evaluated based on contact sequence.
 3. **Invalid State:** NaN values in `data.qpos` or `data.qvel` .
 - Indicates simulation instability; episode terminates with `success = False`.

Success Criteria

An episode is considered **successful** if **all** of the following hold at episode termination:

1. **Head never touches ground** during the episode (no MuJoCo contact between head geoms and floor at any step).
2. **Left arm contacts first:** $t_A^L < t_H$ (from MuJoCo array).
3. **Right arm contacts first:** $t_A^R < t_H$ (from MuJoCo array).
4. **Arms synchronized:** $|t_A^L - t_A^R| \leq 0.1$ seconds, where times are from actual contact detection.

(All contact timing is based on MuJoCo contact array detection only; no feet/knee contact requirements since legs are locked.)

Training Stopping Condition

Training halts early when:

- Success rate $\geq 95\%$ is achieved over a 10-episode rolling window, **AND**
- That $\geq 95\%$ success rate is **maintained for at least 100 consecutive additional episodes**.

Initialization & Reset

Episode Reset (`env.reset()`)

1. Reset all joint positions and velocities to zero (via `mujoco.mj_resetData()`).

2. Raise COM slightly (+0.05 m in Z) to avoid initial collision.
3. Apply an initial push impulse:
 - Fixed push force: **90.0 N** in +X direction (forward).
 - Applied for 5 steps (~0.025 s) at episode start.
4. Set fixed joints explicitly to zero (head pan/tilt, all hip/knee/ankle joints).
5. Clear contact timers: $t_A^L, t_A^R, t_H \leftarrow \infty$.

Reset Returns

- Initial observation (19-dim).
-

Geometry & Collision

Key Collision Geoms

Category	OP3 Geom Names	Purpose
Head	h1c , h2c , h21c , h22c	Head collision meshes
Arms (Left)	la1c , la2c , la3c	Left arm links
Arms (Right)	ra1c , ra2c , ra3c	Right arm links
Legs (Left)	ll1c – ll6c	Left leg link meshes (rigid; not actuated)
Legs (Right)	rl1c – rl6c	Right leg link meshes (rigid; not actuated)
Body	bodyc , body1c – body4c	Torso collision meshes
Floor	floor (world geom)	Ground plane

Contact Detection Algorithm

All contact timing and detection must use MuJoCo contact array
(`data.contact[0:data.ncon]`).

For each step:

1. Iterate through all contacts: `for i in range(int(data.ncon))`
2. Get contact pair:
`c = data.contact[i]; geom1 = int(c.geom1); geom2 = int(c.geom2)`
3. Determine if floor is involved:
`floor_id = int(mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_GEOM, 'floor'))`
4. Check limb membership: if `geom1 == floor_id` and `geom2` in arm/head set, record contact time for that limb. (Or vice versa for `geom2 == floor_id`.)
5. Record first contact time for each limb; ignore subsequent contacts.

No position-based estimation: Contact is only registered when MuJoCo's contact solver detects a collision.

Implementation Notes

Simulation Timestep

- MuJoCo base timestep: `model.opt.timestep` (default ~0.002 s).
- Per environment step: **5 MuJoCo substeps** (cumulative ~0.01 s per action step).
- Episode length: 200 steps → ~2.0 seconds of wall-clock simulation.

Contact Detection Timing

- Contact detection occurs **after** each `mujoco.mj_step()` call, when `data.contact` is populated.
- Contact times are recorded as the step number (in simulation time) when first contact is detected.
- **Critical:** Use MuJoCo contact array only; never infer contact from position or

height thresholds.

Position Control

OP3 uses position-servo actuators. Each actuator is a proportional controller targeting a desired joint position. The policy outputs normalized actions $[-1, 1]$ which map to control targets via the joint's `ctrlrange`.

Actuator Force / Torque

- `data.actuator_force[actuator_id]` gives the scalar force applied by the actuator at current step.
- For position servo: $\text{force} = k_p \cdot (\text{target} - \text{current_pos})$ (simplified).
- Used for torque penalty: penalizes high actuation effort.

Clipping & Bounds

- Observations clipped to $[-50, +50]$.
- Rewards clipped to $[-100, +100]$.
- Actions clipped to $[-1, +1]$ before mapping to control ranges.

File Structure

```
envs/op3_arm_brace/  
├─ __init__.py          # Package init, exports Op3ArmBraceEnv  
├─ env.py               # Core environment class  
├─ op3.xml              # OP3 model (copy or reference from assets/)  
└─ system_explanation.md # This document
```

Differences from op3_brace

Aspect	OP3 Brace (Full)	OP3 Arm Brace
Actuated Joints	12 (6 arms + 6 legs)	6 (arms only)
Action Dim	12	6
Obs Dim	31	19
Obs Components	Body pose (R/P), angular velocity, 12 joint angles/vels, contact flags	Body pose (R/P), angular velocity, 6 arm angles/vels, contact flags
Fixed Joints	Head, hip yaw, ankles	Head, all legs (hip yaw, pitch, roll, knee, ankle)
Contact Reward	Arm-first, arm-sync, knee timing	Arm-first, arm-sync only (no leg contact)
Success Condition	Arms first + arm sync + no head touch	Arms first + arm sync + no head touch
Task Difficulty	Robot can use legs to stabilize	Arms only; more difficult; pure upper-body bracing

Validation Checklist

Before training:

- ☐ OP3 XML loads without errors; all arm joint names resolve.
- ☐ All leg joints are fixed to zero and cannot be actuated.
- ☐ Observation shape = 19; no NaN/Inf during reset.
- ☐ Action space maps cleanly to 6 arm control targets.
- ☐ Contact detection logic identifies floor collisions correctly.
- ☐ First episode runs for 200 steps without crashes.

- ☐ Reward components compute without NaN.
 - ☐ Success flag evaluates correctly after episode.
 - ☐ Network forward pass and policy sampling work on CPU.
-

References

- Baseline: `scripts/brace_only_single/brace_only_single.py`
- Full OP3 Brace: `envs/op3_brace/`
- OP3 Model: `assets/robotis_op3/op3.xml`
- MuJoCo Python API: <https://mujoco.readthedocs.io/>